

Guide de Validation — Applications Web Distribuées

Microservices Spring Boot · Questions & Réponses par critère

Architecture globale du projet

La plateforme est une application éducative de gestion des réclamations et demandes de rattrapage. Elle est construite en architecture microservices avec les composants suivants :

Service	Technologie	Port	Rôle
eureka-server	Spring Boot	8761	Serveur de découverte
gateway-service	Spring Cloud Gateway	8080	Point d'entrée unique + CORS
auth-service	Spring Boot + MySQL	8083	Authentification JWT + profils
claim-service	Spring Boot + MySQL	8081	Gestion des réclamations
retake-service	Spring Boot + MySQL	8082	Demandes de rattrapage
ml-service	Python FastAPI	5000	Prédiction ML (technologie avancée)
Frontend	Angular 17	4200	Interface utilisateur

1. Micro-service individuel (Spring Boot)

1,5 pts

Q : C'est quoi votre microservice individuel ?

R : Mon microservice individuel est le retake-service. Il gère les demandes de rattrapage des étudiants : création, suivi du statut (PENDING / APPROVED / DENIED), upload de justificatifs, et communication avec le claim-service via Feign Client.

✓ [Montrer RetakeRequestController.java et RetakeRequestService.java](#)

Q : Comment votre microservice communique avec les autres ?

R : Le retake-service utilise Feign Client pour appeler le claim-service. Il récupère les détails d'une réclamation et peut lier une demande de rattrapage à une réclamation existante.

```
@FeignClient(name = "claim-service") public interface ClaimClient {  
    @GetMapping("/api/claims/{id}") ClaimDTO getClaimById(@PathVariable Long id); }
```

2. Compréhension du code

3 pts

Q : Expliquez le flux complet quand un étudiant crée une réclamation.

R : 1. L'étudiant remplit le formulaire Angular et soumet. 2. Le frontend appelle POST /api/claims via le gateway (port 8080). 3. Le gateway route vers claim-service (port 8081). 4. Le claim-service vérifie que l'étudiant n'a pas déjà une réclamation identique (anti-doublon). 5. Il sauvegarde la réclamation en base MySQL et retourne la réponse. 6. L'admin reçoit la réclamation et peut répondre ou créer une demande de rattrapage liée.

✓ [Montrer ClaimController.java → ClaimService.java → ClaimRepository.java](#)

Q : Comment fonctionne le JWT dans votre projet ?

R : À la connexion (POST /auth/login), l'auth-service génère un token JWT signé avec une clé secrète. Ce token contient le nom, le rôle (STUDENT/ADMIN) et l'ID de l'utilisateur. Chaque requête suivante envoie ce token dans le header Authorization: Bearer . Le JwtAuthFilter de l'auth-service le vérifie à chaque appel.

```
Authorization: Bearer eyJ0eXAiOiJKV1Qi...
```

Q : Comment les rôles sont gérés ?

R : Deux rôles : STUDENT et ADMIN. Le rôle est stocké dans le token JWT. Côté frontend Angular, on lit le rôle depuis le token pour afficher ou cacher des fonctionnalités. Exemple : seul l'ADMIN voit les boutons Approve/Deny et la colonne ML Analysis.

✓ *Montrer le token décodé et le composant `retake-requests.component.ts` ligne `isAdmin`*

3. Microservice technologie avancée + BD

2 pts

Q : Quel est votre microservice avec technologie avancée ?

R : Le ml-service développé en Python avec FastAPI. C'est une technologie différente de Spring Boot — c'est le changement de technologie demandé. Il expose une API REST de Machine Learning qui prédit la réussite des étudiants.

✓ *1 pt pour le changement de technologie (Python/FastAPI au lieu de Java/Spring)*

Q : Quelle base de données utilise ce microservice ?

R : Le ml-service ne stocke pas en base — il charge un modèle ML pré-entraîné (fichier .pkl) et retourne des prédictions en temps réel. Les autres microservices utilisent MySQL. Si la question porte sur la BD avancée : le retake-service utilise MySQL avec des relations JPA.

■ **Si le validateur insiste sur MongoDB/PostgreSQL : expliquer que le choix de FastAPI comme technologie avancée compense.**

Q : Comment le ml-service est intégré avec Spring ?

R : Le gateway Spring Cloud route /api/ml/** vers le ml-service (Python port 5000) avec StripPrefix=2. Le frontend Angular appelle le gateway normalement — il ne sait même pas que c'est du Python derrière.

```
/api/ml/analyze → gateway (8080) → ml-service:5000/analyze
```

4. Serveur de découverte (Eureka)

1 pt

Q : C'est quoi Eureka et pourquoi vous l'utilisez ?

R : Eureka est un serveur de découverte de services (Netflix OSS). Chaque microservice s'enregistre automatiquement au démarrage avec son nom et son adresse. Le gateway et le Feign Client utilisent Eureka pour trouver les services sans hardcoder les URLs.

✓ *Montrer <http://localhost:8761> — dashboard Eureka avec tous les services enregistrés*

Q : Comment un service s'enregistre sur Eureka ?

R : Dans `application.properties` de chaque service :
`eureka.client.service-url.defaultZone=http://eureka-server:8761/eureka/` Et `@EnableEurekaServer` sur la classe principale du eureka-server.

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

5. Config Server

1 pt

Q : Avez-vous un Config Server ?

R : Non, nous n'avons pas implémenté de Config Server Spring Cloud. Chaque microservice a son propre application.properties. C'est un point à améliorer — le Config Server permettrait de centraliser toutes les configurations.

■ Mentionner que Docker Compose compense partiellement en centralisant les variables d'environnement.

✗ Point non implémenté — à signaler honnêtement au validateur.

6. Gateway

1 pt

Q : C'est quoi votre gateway et comment il fonctionne ?

R : Spring Cloud Gateway — c'est le point d'entrée unique de toute l'application. Le frontend n'appelle que localhost:8080. Le gateway route chaque requête vers le bon microservice selon le chemin URL et gère aussi le CORS pour le frontend Angular.

✓ Montrer application.yml du gateway avec les routes configurées

Q : Montrez les routes configurées.

R : /auth/**, /user/**, /api/students/** → auth-service:8083 /api/claims/** → claim-service:8081 /api/retake-requests/** → retake-service:8082 /api/ml/** → ml-service:5000 (avec StripPrefix=2)
/api/ml/analyze → StripPrefix=2 → ml-service:5000/analyze

7. Sécurité

2 pts — max 1 pt
sans Keycloak

Q : Vous utilisez Keycloak ?

R : Non. Nous utilisons JWT (JSON Web Token) implémenté manuellement dans l'auth-service avec Spring Security. C'est une approche de sécurité complète et fonctionnelle, même si différente de Keycloak.

■ Selon la grille : sans Keycloak → note entre 0.5 et 1 selon l'effort. Montrer l'effort fourni.

Q : Comment fonctionne votre sécurité concrètement ?

R : 1. POST /auth/login → auth-service vérifie email/password → génère JWT signé (HS512) 2. Le token est stocké côté client (localStorage) 3. Chaque requête envoie le token dans Authorization: Bearer 4. JwtAuthFilter intercepte et valide le token avant chaque endpoint sécurisé 5. Les routes /auth/login et /auth/register sont publiques, tout le reste est protégé

✓ Montrer JwtAuthFilter.java et SecurityConfig.java — c'est une sécurité réelle, complète

Q : Comment les rôles sont protégés côté backend ?

R : Dans SecurityConfig.java, les endpoints sont protégés par .anyRequest().authenticated(). Côté frontend, les composants vérifient le rôle depuis le token JWT décodé. Un STUDENT ne voit pas les boutons d'administration et ne peut pas accéder aux routes admin.

✓ Insister sur : JWT complet + Spring Security configuré + gestion des rôles côté front

8. Git et documentation

1 pt

Q : Montrez votre historique Git.

R : Nous avons des commits réguliers et bien nommés avec la convention Conventional Commits : feat: nouvelle fonctionnalité | fix: correction | chore: maintenance Exemples : • feat(ml): add Python FastAPI ML service • fix(cors): remove duplicate @CrossOrigin annotations • feat(docker): add Dockerfiles and docker-compose • feat(auth): add StudentAcademicProfile entity and REST API

✓ *Faire git log --oneline pour montrer l'historique*

9. Docker Compose

2 pts

Q : Votre docker-compose lance quoi ?

R : Tous les services en une seule commande : MySQL, Eureka, Gateway, Auth, Claim, Retake, ML Service. Les services dépendants attendent que MySQL et Eureka soient healthy avant de démarrer (grâce aux healthchecks et depends_on).

✓ *Montrer docker-compose.yml et faire docker ps pour prouver que tout tourne*

Q : Comment les microservices se trouvent dans Docker ?

R : Ils communiquent via le réseau Docker 'platform-network'. Chaque service utilise le nom du conteneur comme hostname (ex: auth-service:8083). De l'extérieur, seul le gateway (8080) est exposé — les autres ports sont internes.

`docker-compose up --build -d` → tout démarre automatiquement

10. Partie Frontend

1,5 pt

Q : C'est quoi votre frontend ?

R : Angular 17 avec Angular Material. Il communique uniquement avec le gateway (localhost:8080). Il a deux rôles distincts : STUDENT (créer des réclamations, voir ses demandes) et ADMIN (gérer toutes les réclamations, approuver/refuser les rattrapages, voir l'analyse ML).

✓ *Montrer la page Retake Requests avec la colonne ML Analysis en tant qu'ADMIN*

11. Communication entre Microservices

2 pts

Q : Comment vos microservices communiquent entre eux ?

R : Via Feign Client (communication synchrone REST). Le retake-service appelle le claim-service pour récupérer les détails d'une réclamation et pour lier une demande de rattrapage à une réclamation.

✓ *Montrer ClaimClient.java dans retake-service*

Q : Montrez un scénario de communication concret.

R : Scénario 1 : Quand un admin approuve une demande de rattrapage, le retake-service appelle claim-service via Feign pour mettre à jour le statut de la réclamation liée. Scénario 2 : Quand on affiche le détail d'une demande de rattrapage, le retake-service appelle claim-service pour récupérer les infos de la réclamation originale.

✓ *2 scénarios distincts → score maximal sur ce critère*

Q : Pourquoi Feign Client et pas RestTemplate ?

R : Feign Client est plus moderne, plus lisible et s'intègre nativement avec Eureka. On définit juste une interface avec les annotations — Feign génère le code HTTP automatiquement. RestTemplate est plus verbeux et déprécié dans les nouvelles versions de Spring.

12. Valeurs Ajoutées

2 pts

Q : Quelles sont vos valeurs ajoutées ?

R : Plusieurs valeurs ajoutées significatives : 1. ML Service Python/FastAPI intégré : prédiction de réussite des étudiants via Random Forest + K-Means, exposé via gateway et affiché en temps réel dans l'interface. 2. AI Draft Response : intégration avec l'API Groq (LLM) pour générer automatiquement des réponses aux réclamations, aidant les admins à rédiger plus vite. 3. Système de messagerie interne : chat en temps réel entre étudiants et admins dans le contexte d'une réclamation. 4. Vérification de documents : upload et vérification de justificatifs pour les demandes de rattrapage.

✓ 4 valeurs ajoutées distinctes → viser les 2 pts complets

Q : Pourquoi pas RabbitMQ ou communication asynchrone ?

R : Notre communication synchrone via Feign Client est adaptée à notre besoin : les réponses doivent être immédiates (l'utilisateur attend le résultat). RabbitMQ serait pertinent pour des notifications asynchrones — c'est une amélioration possible.

■ Si le validateur insiste : reconnaître que c'est une amélioration future envisagée.

Récapitulatif des points

Critère	Max	Notre situation
Microservice individuel (Spring Boot)	1,5	✓ retake-service complet
Compréhension du code	3	✓ flux, JWT, rôles expliqués
Technologie avancée + BD	2	✓ Python FastAPI (1pt) + MySQL (1pt)
Eureka	1	✓ dashboard visible sur :8761
Config Server	1	✗ non implémenté
Gateway	1	✓ Spring Cloud Gateway + CORS
Sécurité (JWT, pas Keycloak)	2	■ max ~1pt — JWT complet
Git et documentation	1	✓ commits réguliers et conventionnels
Docker Compose	2	✓ tous les services dockerisés
Frontend Angular	1,5	✓ deux rôles, ML Analysis
Communication Feign Client	2	✓ 2 scénarios démontrés
Valeurs ajoutées	2	✓ ML + AI + Messagerie + Documents
TOTAL estimé	20	~15-17 / 20